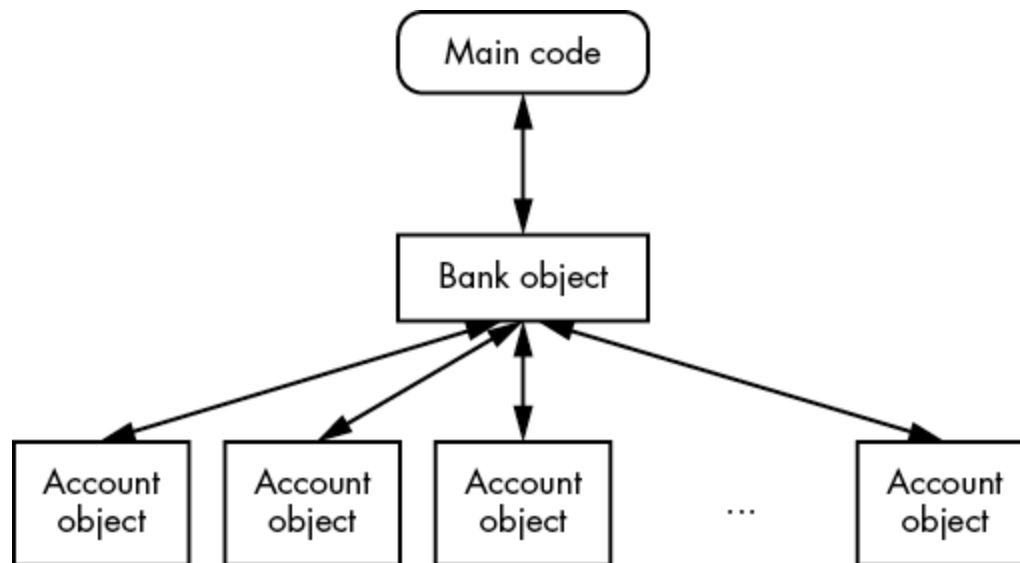


To create this hierarchy, we need some main code that handles the highest-level menuing system. In response to a choice of action, the main code will call a method of the `Bank` object (for example, `deposit()` or `withdraw()`). The `Bank` object will gather the information it needs (account number, password, amount to deposit or withdraw), reach into its dictionary of accounts to find the matching user account, and call the appropriate method for that user's account.



*Figure 4-1: The main code manages a `Bank` object, which manages many `Account` objects.*

There are three layers in this division of labor:

1. The main code that creates and talks to a single `Bank` object
2. The `Bank` object that manages a dictionary of `Account` objects and calls methods of those objects
3. The `Account` objects themselves

With this approach, we only have a single global variable, the `Bank` object. In fact, the main code has no idea that `Account` objects even exist. Conversely, each `Account` object has no clue (and does not care) what the top-level user interface of the program is. The `Bank` object receives messages from the main code and communicates with the appropriate `Account` object.

The key advantage of this approach is that we have broken down a much larger program into smaller subprograms: in this case, the main code and